

ARTI2001 MACHINE LEARNING – MODULE 1 CHEAT SHEET

Topics 1–4: mathematical foundations, linear regression + MLE, perceptron + logistic regression, SVM + kernels

TOPIC 1 – ML FOUNDATIONS + MATHS

Learning paradigms

Supervised	Labelled pairs. Regression predicts continuous values; classification predicts classes.
Unsupervised	Finds structure in unlabelled data: clustering, dimensionality reduction, association rules.
Reinforcement	Agent acts, receives rewards/penalties, learns a reward-maximising policy.
Semi-supervised	Small labelled set + larger unlabelled set.
Self-supervised	Learning targets are constructed from the data itself.
Linear algebra	

$$x^\top y = \sum_i x_i y_i, \quad \|x\|_2 = \sqrt{\sum_i x_i^2}$$

$$(AB)_{ij} = \sum_k A_{ik} B_{kj}, \quad (A^\top)_{ij} = A_{ji}, \quad AA^{-1} = I$$

Dimensions: if $X \in \mathbb{R}^{n \times p}$ and $\beta \in \mathbb{R}^{p \times 1}$, then $X\beta \in \mathbb{R}^{n \times 1}$.

Calculus

$$\frac{d}{dx} x^n = n x^{n-1}, \quad \frac{d}{dx} f(g(x)) = f'(g(x)) g'(x)$$

$$\frac{d}{dx} e^x = e^x, \quad \frac{d}{dx} \ln x = \frac{1}{x}$$

$$\nabla f = \begin{bmatrix} \partial f / \partial x_1 & \cdots & \partial f / \partial x_d \end{bmatrix}^\top$$

Gradient points toward steepest increase, so gradient descent moves opposite it:

$$\theta \leftarrow \theta - \eta \nabla L(\theta)$$

Probability + statistics

$$E[X] = \sum_x x P(X = x), \quad \text{Var}(X) = E[X^2] - E[X]^2, \quad \sigma_X = \sqrt{\text{Var}(X)}$$

Normal standardisation:

$$Z = \frac{X - \mu}{\sigma}$$

For an interval:

$$P(a < X < b) = \Phi\left(\frac{b - \mu}{\sigma}\right) - \Phi\left(\frac{a - \mu}{\sigma}\right)$$

Bayes:

$$P(A \mid B) = \frac{P(B \mid A)P(A)}{P(B)}$$

Total probability:

$$P(A) = \sum_i P(A \mid B_i) P(B_i)$$

Total expectation / variance:

$$E[X] = E_Y[E[X \mid Y]] \\ \text{Var}(X) = E_Y[\text{Var}(X \mid Y)] + \text{Var}_Y(E[X \mid Y])$$

Tutorial 1 quick answers

$$\begin{array}{ll} \frac{d}{dx} (3x^2 - 2x + 5) & 6x - 2 \\ \frac{d}{dx} \ln(x^2 + 1) & \frac{2x}{x^2 + 1} \\ X & \sim Z \in (-1, 2) \Rightarrow 81.86\% \end{array}$$

$$\begin{array}{ll} N(150, 15^2), 135 & < \\ X < 180 & \\ -1.5 < Z < 2 & 91.04\% \\ P(X) = .1, .2, .4, .2, .1 & E = 3, \text{ Var} = 1.2, \sigma \approx 1.095 \\ \text{on } 1, \dots, 5 & \\ \text{Cancer screen} & P(+) = 0.9(0.01) + 0.03(0.99) = 0.0387; P(C \mid +) = 0.009 / 0.0387 \approx 0.2326 \end{array}$$

Pipeline, fit + leakage

Pipeline: prepare $\rightarrow$ train $\rightarrow$ validate/tune $\rightarrow$ final test once $\rightarrow$ deploy/monitor.

Underfitting / high bias: training and unseen performance both poor.

Overfitting / high variance: training strong but unseen performance much worse.

Leakage: split first. Learn imputation, encoding, scaling, etc. on training data only, then reuse those fitted transforms on validation/test.

Scaling especially matters for **distance-, margin- and gradient-based** models.

TOPIC 2 – LINEAR REGRESSION + MLE

Model, residual + cost

Simple linear model:

$$\hat{y}_i = w x_i + b$$

Residual:

$$r_i = y_i - \hat{y}_i$$

Half-MSE objective:

$$J(w, b) = \frac{1}{2m} \sum_{i=1}^m (y_i - \hat{y}_i)^2$$

Why square errors? Prevent cancellation, penalise large errors, and obtain a smooth differentiable objective.

Ordinary squared-error linear regression has a convex quadratic cost surface.

Gradient descent

For $\hat{y}_i = w x_i + b$:

$$\frac{\partial J}{\partial w} = \frac{1}{m} \sum_i (\hat{y}_i - y_i) x_i, \quad \frac{\partial J}{\partial b} = \frac{1}{m} \sum_i (\hat{y}_i - y_i)$$

$$w \leftarrow w - \alpha \frac{\partial J}{\partial w}, \quad b \leftarrow b - \alpha \frac{\partial J}{\partial b}$$

Update parameters **simultaneously using old values**. Small α : slow. Large α : may overshoot/diverge.

Gaussian noise: MLE $\iff$ SSE

Assume

$$y_i \mid x_i \sim \mathcal{N}(w_0 + w_1 x_i, \sigma^2),$$

with independent observations and fixed σ^2 .

Then

$$\log L(w_0, w_1) = -\frac{N}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} \sum_{i=1}^N [y_i - (w_0 + w_1 x_i)]^2$$

Define

$$\text{SSE} = \sum_i [y_i - (w_0 + w_1 x_i)]^2.$$

The first term is constant in w_0, w_1 , and the SSE is multiplied by a negative constant, so

$$\arg \max \log L = \arg \min \text{SSE}$$

Polynomial regression in matrix form

For

$$y = a x^2 + b x + c,$$

write

$$Y = X\beta$$

with

$$Y = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}, \quad X = \begin{bmatrix} x_1^2 & x_1 & 1 \\ x_2^2 & x_2 & 1 \\ \vdots & \vdots & \vdots \\ x_n^2 & x_n & 1 \end{bmatrix}, \quad \beta = \begin{bmatrix} a \\ b \\ c \end{bmatrix}.$$

Normal equation

$$\hat{\beta} = (X^\top X)^{-1} X^\top Y$$

Inverse-form solution requires $X^\top X$ invertible. In practice, QR/SVD/pseudoinverse solvers are more stable than explicitly forming the inverse.

	Gradient descent	Normal equation
Learning rate	Yes	No
Iterative	Yes	No
Many features	Usually better	Inverse step costly
Scaling	Recommended	Not required by formula

Feature scaling

Z-score feature scaling:

$$x' = \frac{x - \mu}{\sigma}$$

Very different feature ranges produce elongated contours and slow zig-zag descent. Fit μ, σ on training data only.

Evaluation

$$\text{MSE} = \frac{1}{m} \sum_i (y_i - \hat{y}_i)^2$$

$$\text{RMSE} = \sqrt{\text{MSE}}$$

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$$

MSE is in y^2 units; RMSE is in original y units. R^2 measures improvement over predicting the mean and can be negative on poor unseen predictions.

Assumptions + traps

Approximate linearity; independence; homoscedasticity; residual normality mainly for inference; no severe multicollinearity in multiple regression.

Trap: Residual is actual – predicted, but gradient formulas often use predicted – actual.

Trap: MLE means maximise likelihood/log-likelihood; with fixed Gaussian variance this becomes minimise SSE.

TOPIC 3 – PERCEPTRON + LOGISTIC REGRESSION

Shared linear score

$$z = w^\top x + b$$

The decision boundary is

$$w^\top x + b = 0.$$

Changing w changes orientation; changing b shifts the boundary.

Perceptron

Labels: $y_i \in \{-1, +1\}$.

Score:

$$s_i = w^\top x_i + b, \quad \hat{y}_i = \text{sign}(s_i).$$

Signed score:

$$y_i s_i$$

Course convention:

$$\begin{array}{l} y_i s_i > 0 \Rightarrow \text{correct, no update} \\ y_i s_i \leq 0 \Rightarrow \text{mistake/tie, update} \end{array}$$

Update:

$$w \leftarrow w + \eta y_i x_i, \quad b \leftarrow b + \eta y_i$$

Perceptron criterion:

$$L_i = \max(0, -y_i s_i)$$

Sequential training: the parameters after example i become the starting parameters for example $i + 1$.

Logistic regression

Labels: $y_i \in \{0, 1\}$.

$$p = \sigma(z) = \frac{1}{1 + e^{-z}}$$

where $p = P(y = 1 \mid x)$. Since $\sigma(0) = 0.5$,

$$p \geq 0.5 \iff z \geq 0.$$

Therefore logistic regression has the same linear boundary $z = 0$.

Binary cross-entropy

$$L(y, p) = -[y \log p + (1 - y) \log(1 - p)]$$

If $y = 1$: $L = -\log p$. If $y = 0$: $L = -\log(1 - p)$. Confidently wrong predictions get very large loss.

The key gradient: $p - y$

For each example:

$$\frac{\partial L_i}{\partial z_i} = p_i - y_i$$

$$\frac{\partial L_i}{\partial w} = (p_i - y_i)x_i, \qquad \frac{\partial L_i}{\partial b} = p_i - y_i$$

Batch averages:

$$\frac{\partial J}{\partial w} = \frac{1}{m} \sum_i (p_i - y_i)x_i, \qquad \frac{\partial J}{\partial b} = \frac{1}{m} \sum_i (p_i - y_i)$$

Update:

$$w \leftarrow w - \eta \frac{\partial J}{\partial w}, \qquad b \leftarrow b - \eta \frac{\partial J}{\partial b}$$

Interpretation:

$$\begin{aligned} p - y < 0 : \quad & p < y \Rightarrow \text{raise score} \\ p - y > 0 : \quad & p > y \Rightarrow \text{lower score} \end{aligned}$$

A sufficiently small step opposite the gradient reduces loss locally because $-\nabla J$ is the direction of steepest decrease.

Perceptron vs logistic

	Perceptron	Logistic
Labels	$-1, +1$	$0, 1$
Output	$\text{sign}(z)$	$\sigma(z)$
Signal	yz	$p - y$
Update	Mistake/tie only	Nonzero gradient
Boundary	$z = 0$	$z = 0$

Tutorial 3 worked answers

Q1: $w = [1, -1], b = 0, \eta = 1$.

Example 1: $x_1 = [2, 3], y_1 = +1$

$$s_1 = 1(2) + (-1)(3) = -1$$

so update:

$$w = [1, -1] + [2, 3] = [3, 2], \qquad b = 1.$$

Example 2: $x_2 = [-1, -2], y_2 = -1$

$$s_2 = 3(-1) + 2(-2) + 1 = -6, \qquad y_2 s_2 = 6 > 0.$$

No update, so

$$w = [3, 2], \quad b = 1.$$

Q2:

$$p_1 = \sigma(0.5) = 0.62246, \qquad p_2 = \sigma(-0.3) = 0.42556$$

$$\frac{\partial J}{\partial w} \approx -0.15822, \qquad \frac{\partial J}{\partial b} \approx 0.02401$$

with $\eta = 0.1$:

$$w_{\text{new}} \approx 1.01582, \quad b_{\text{new}} \approx -0.00240.$$

Q3:

$$z = 2x_1 - x_2 - 3 \Rightarrow x_2 = 2x_1 - 3.$$

$$A = (2, 2) : z = -1, \quad p = 0.2689, \text{ class } 0$$

$$B = (1, 5) : z = -6, \quad p = 0.00247, \text{ class } 0$$

$$C = (3, 1) : z = 2, \quad p = 0.8808, \text{ class } 1$$

Most confident: B , because p is farthest from 0.5.

Trap: Do not mix label conventions: perceptron/SVM use ± 1 ; logistic uses 0/1.

Trap: In sequential perceptron work, carry the updated w, b into the next example.

TOPIC 4 – SUPPORT VECTOR MACHINES

Score, distance + margin

Linear score:

$$f(x) = w^\top x + b$$

Signed geometric distance:

$$\frac{f(x)}{\|w\|}$$

Unsigned distance:

$$\frac{|f(x)|}{\|w\|}$$

Labelled geometric margin:

$$\gamma_i = \frac{y_i f(x_i)}{\|w\|}$$

Keep distinct: raw score $f(x)$, geometric distance $f/\|w\|$, and margin score $yf(x)$.

Canonical scaling

$$y_i f(x_i) \geq 1$$

One-sided margin:

$$\frac{1}{\|w\|}$$

Full margin width:

$$\frac{2}{\|w\|}$$

Smaller $\|w\| \Rightarrow$ wider geometric margin.

Interpret $yf(x)$:

$$\begin{aligned} yf > 0 : & \text{correct side,} & yf = 0 : & \text{boundary} \\ 0 < yf < 1 : & \text{correct but inside margin} \\ yf = 1 : & \text{support hyperplane,} & yf < 0 : & \text{misclassified.} \end{aligned}$$

Soft margin, slack + C

$$\begin{aligned} \xi_i = 0 & \quad \text{on/outside required margin} \\ 0 < \xi_i < 1 & \quad \text{inside margin, still correctly classified} \\ \xi_i = 1 & \quad \text{on decision boundary} \\ \xi_i > 1 & \quad \text{misclassified} \end{aligned}$$

Small C: violations penalised less $\Rightarrow$ wider/more regularised margin may be preferred.

Large C: violations heavily penalised $\Rightarrow$ training fit matters more, often with a narrower margin.

A wider margin may improve robustness to small perturbations, but does not guarantee better test performance.

Support vectors + dual view

Support vectors are influential, boundary-defining examples.

In the soft-margin dual:

$$0 \leq \alpha_i \leq C$$

$$\alpha_i > 0 \Rightarrow \text{example } i \text{ influences the solution.}$$

Kernel trick

$$K(x, z) = \phi(x)^\top \phi(z)$$

The kernel computes the transformed-space dot product **without explicitly constructing all transformed features**.

It does **not** guarantee cheap training on a large dataset; pairwise kernel evaluations and optimisation can still be expensive.

Kernel	Form / behaviour
Linear	$K(x, z) = x^\top z$; straight boundary; tune C
Polynomial	$K = (\gamma x^\top z + r)^d$; global curve; tune C, d, γ, r
RBF	$K = \exp(-\gamma \ x - z\ ^2)$; local curve; tune C, γ
RBF γ	small $\rightarrow$ smoother; large $\rightarrow$ more local/complex

Scale features before comparing polynomial/RBF SVMs; cross-validate on training data and test the final selected model once.

Polynomial-kernel identity

Tutorial feature map:

$$\phi(x_1, x_2) = \left(x_1^2, \sqrt{2} x_1 x_2, x_2^2 \right)$$

Then

$$\begin{aligned} \phi(x)^\top \phi(y) &= x_1^2 y_1^2 + 2x_1 x_2 y_1 y_2 + x_2^2 y_2^2 \\ &= (x_1 y_1 + x_2 y_2)^2 \\ &= \boxed{(x^\top y)^2}. \end{aligned}$$

Tutorial 4 worked answers

Given

$$f(x) = x_1 + x_2 - 3, \qquad w = (1, 1), \qquad \|w\| = \sqrt{2}.$$

$$(1, 2, +1) : \quad f = 0, \quad \frac{f}{\|w\|} = 0, \quad yf = 0$$

boundary; margin violation.

$$(2, 3, +1) : \quad f = 2, \quad \frac{f}{\|w\|} = \sqrt{2}, \quad yf = 2$$

correct; no violation.

$$(3, 3, -1) : \quad f = 3, \quad \frac{f}{\|w\|} = \frac{3}{\sqrt{2}}, \quad yf = -3$$

misclassified; violation.

$$(1, 1, -1) : \quad f = -1, \quad \frac{f}{\|w\|} = -\frac{1}{\sqrt{2}}, \quad yf = 1$$

correct; on support hyperplane.

For $w = (3, 4)$:

$$\|w\| = 5, \quad \boxed{\text{one side} = 0.2, \quad \text{full width} = 0.4}.$$

Small C favours the wider-margin model with violations; large C favours the zero-violation model.

For the tutorial feature map:

$$\phi(1, 1) = (1, \sqrt{2}, 1), \qquad \phi(2, 2) = (4, 4\sqrt{2}, 4).$$

$$\phi(1, 1)^\top \phi(2, 2) = 4 + 8 + 4 = 16.$$

Trap: $f(x)$ is not geometric distance until divided by $\|w\|$.

Trap: Correct classification does not imply satisfying the margin: $0 < yf < 1$ is correct but still a margin violation.

Trap: One-sided margin $= 1/\|w\|$; full width $= 2/\|w\|$.

Trap: Large C does not mean a wider margin; it prioritises reducing violations.

Last-minute priorities

Know these cold:

$$\text{Gaussian MLE} \iff \min \text{SSE}$$

$$\text{Perceptron update if } y(w^\top x + b) \leq 0$$

$$p = \sigma(z), \quad \nabla \text{ signal} = p - y, \quad \text{boundary } z = 0$$

$$\text{SVM distance} = \frac{f}{\|w\|}, \quad yf \geq 1$$

$$\text{small } C : \text{ wider margin, more violations tolerated}$$

$$\text{large } C : \text{ violations penalised strongly}$$